

IT3081 Statistical Modelling From Statistical Testing to Model Building in R

Statistical Modeling Support Group

Purpose. The laboratory sheets are not isolated collections of R commands. Together, they describe a complete workflow for answering a statistical question, building a predictive model, checking whether it is trustworthy, evaluating it on unseen data and interpreting the final result.

Contents

1	The Big Picture	2
1.1	One complete workflow	2
2	Stage 1: Define the Question and Response Type	3
2.1	Inference question versus prediction question	3
3	Stage 2: Import, Inspect and Clean the Data	4
4	Stage 3: Explore the Data and Test Basic Questions	4
4.1	Choosing a test	5
4.2	A reusable testing pattern	5
4.3	ANOVA connects group comparison to regression	5
4.4	Normality and correlation	5
5	Stage 4: Split Data Before Predictive Modelling	6
6	Stage 5: Build a Continuous-Outcome Baseline	6
6.1	Evaluate numerical predictions	7
6.2	Recognise overfitting	7
7	Stage 6: Diagnose the Linear Model	7
8	Stage 7: Control Model Complexity	8
8.1	Subset and stepwise selection	8
8.2	Ridge regression	9
8.3	LASSO	9
8.4	Why cross-validation is central	9
9	Stage 8: Change the Model When the Response Changes	10
9.1	Binary response: logistic regression	10
10	Stage 9: Compare Classification Models	11
10.1	LDA and QDA	11
10.2	Naive Bayes	11
10.3	Confusion matrix and accuracy	11

11 Stage 10: Match the Model to Special Response Types	11
11.1 Multinomial logistic regression	11
11.2 Ordinal logistic regression	12
11.3 Poisson regression for counts	12
11.4 Exponential regression through transformation	12
11.5 Gamma regression for positive continuous responses	13
12 How the Early Skills Support Later Models	13
13 A Practical End-to-End Example	14
14 Master Model-Selection Guide	14
15 Common Mistakes to Avoid	15
16 Reusable R Skeleton	15
17 Final Practical Checklist	16
18 Core Formula Summary	17

1 The Big Picture

The labs follow a natural learning sequence:

Lab	Main skill	Role in a real analysis
02	Statistical tests and relationships	Understand the data, compare groups, examine assumptions and identify evidence of association.
03	Linear regression and feature selection	Build a continuous-outcome model, diagnose its assumptions and simplify its predictors.
04	Ridge and LASSO	Control model complexity, use cross-validation and compare performance on unseen data.
05	Binary logistic regression	Model the probability of a two-category outcome and interpret odds ratios.
06	LDA, QDA and Naive Bayes	Classify observations and compare alternative classification rules.
07	Specialised regression models	Match the model to unordered, ordered, count and positive continuous responses.

Key idea. The main question is not which R function should be memorised. The main question is: what type of response do we have, what evidence do we need, and how will the fitted model be evaluated?

1.1 One complete workflow

1. Define the practical question and identify the response variable.
2. Import, inspect and clean the data.
3. Explore distributions, groups and relationships.

4. Use appropriate statistical tests when an inferential question is present.
5. Divide data into training and testing sets for predictive modelling.
6. Fit a simple baseline model.
7. Check assumptions or distributional suitability.
8. Improve or simplify the model using feature selection or regularisation.
9. Predict unseen observations and calculate an evaluation measure.
10. Interpret the model in the original application.

2 Stage 1: Define the Question and Response Type

The response variable determines the family of suitable models.

Response type	Example from the labs	Suitable method
Continuous, approximately symmetric	Boston median house value	Linear regression
Continuous with many predictors	Car fuel economy	MLR, ridge or LASSO
Binary	Leukaemia remission: yes/no	Binary logistic regression
Unordered classes	Three Iris species	LDA, QDA, Naive Bayes or multinomial logistic
Ordered classes	Housing satisfaction	Ordinal logistic regression
Count	Number of yarn breaks	Poisson regression
Positive continuous, right-skewed	Assault measure	Gamma regression
Positive exponential relationship	Puromycin reaction rate	Log-linear exponential regression

2.1 Inference question versus prediction question

Inference asks whether an effect or difference is supported by the data. Examples are:

- Is a sample mean different from 15?
- Do two groups have different means or proportions?
- Are several group means equal?
- Is a regression coefficient statistically significant?

Prediction asks how accurately a fitted model predicts new observations. Examples are:

- How accurately can house value be predicted?
- Which model best classifies a new Iris flower?
- Does ridge regression reduce test RMSE?

Practical meaning. A small p -value does not guarantee good predictive performance, and a low test error does not by itself explain why a relationship exists. Inferential evidence and predictive evaluation answer different questions.

3 Stage 2: Import, Inspect and Clean the Data

Every model-building task should begin with data inspection.

```
# Import an external file
dat <- read.csv("data.csv")
# or
dat <- read.table("data.txt", header = TRUE)

# Understand the data
head(dat)
str(dat)
dim(dat)
summary(dat)
colSums(is.na(dat))

# Remove incomplete observations when appropriate
dat_clean <- na.omit(dat)
```

Inspection answers several practical questions:

- Is the response numeric, binary, categorical, ordered or a count?
- Are categorical variables stored as factors?
- Are units and variable names correct?
- Are values missing, impossible or duplicated?
- Is the sample size large enough for the proposed model?

For a categorical variable:

```
dat$group <- factor(dat$group)
levels(dat$group)
contrasts(dat$group)
```

R uses the first factor level as the reference category unless the analyst changes it. The interpretation of categorical coefficients always depends on that reference.

4 Stage 3: Explore the Data and Test Basic Questions

Lab 02 supplies the tools used before and around later modelling tasks. These tools describe the data and test focused scientific questions.

4.1 Choosing a test

Question	Condition	R function
One mean versus a target	Population SD unknown	<code>t.test(x, mu = ...)</code>
Two independent means	Two groups	<code>t.test(x, y, ...)</code>
One or two means	Population SD known	<code>BSDA::z.test()</code>
Two variances	Independent samples	<code>var.test(x, y)</code>
One proportion	Success count and sample size	<code>prop.test()</code>
Two proportions	Two success counts and sizes	<code>prop.test()</code>
More than two means	One categorical grouping variable	<code>aov()</code>
Equality of group variances	Before or with ANOVA	<code>car::leveneTest()</code>
Pairwise differences after ANOVA	Significant overall ANOVA	<code>agricolae::LSD.test()</code>
Normality	Numeric sample or residuals	<code>shapiro.test()</code>
Linear association	Two numeric variables	<code>cor()</code> and <code>cor.test()</code>

4.2 A reusable testing pattern

Every hypothesis test should be handled in the same order:

1. state H_0 and H_1 ;
2. choose the correct test and significance level α ;
3. run the R function;
4. compare the p -value with α ;
5. state the decision and the practical conclusion.

```
# Examples from Lab 02
t.test(my_data, mu = 15)
t.test(group1, group2, var.equal = TRUE)
BSDA::z.test(data, mu = 100, sigma.x = 15)
var.test(group1, group2)
prop.test(x = 60, n = 100, p = 0.5)
prop.test(x = c(60, 80), n = c(100, 200))
```

4.3 ANOVA connects group comparison to regression

One-way ANOVA tests

$$H_0 : \mu_1 = \mu_2 = \dots = \mu_g$$

against the alternative that at least one group mean differs.

```
anova_fit <- aov(score ~ technique, data = df)
summary(anova_fit)
car::leveneTest(score ~ technique, data = df)
agricolae::LSD.test(anova_fit, "technique")
```

ANOVA and regression are closely related. A categorical predictor in a regression model is represented by dummy variables, and the overall test asks whether the group-related coefficients jointly improve the model.

4.4 Normality and correlation

```
hist(x)
shapiro.test(x)

plot(x, y, pch = 16)
cor(x, y)
cor.test(x, y)
```

Correlation measures the strength and direction of a pairwise linear relationship. It does not establish causation, adjust for other variables or measure out-of-sample predictive performance.

Key idea. Statistical tests should not be used as an automatic predictor-filtering machine. They help explain the data and assumptions. Final predictor choice should also consider subject knowledge, model stability and test-set performance.

5 Stage 4: Split Data Before Predictive Modelling

Training data are used to estimate the model. Test data simulate unseen cases and are used only for final evaluation.

```
set.seed(42)
train_id <- sample(seq_len(nrow(dat)),
                  size = round(0.8 * nrow(dat)))
train <- dat[train_id, ]
test <- dat[-train_id, ]
```

For a continuous response, `caret::createDataPartition()` can create a more balanced split:

```
train_id <- caret::createDataPartition(dat$y,
                                       p = 0.8,
                                       list = FALSE)
train <- dat[train_id, ]
test <- dat[-train_id, ]
```

Practical meaning. All competing predictive models should use the same training and test observations. Otherwise, performance differences may be caused by the split rather than the model.

6 Stage 5: Build a Continuous-Outcome Baseline

For a continuous response, begin with a model that is easy to understand.

$$Y_i = \beta_0 + \beta_1 X_{i1} + \dots + \beta_p X_{ip} + \varepsilon_i.$$

```
library(MASS)
data("Boston")

# Simple and multiple regression
fit_simple <- lm(medv ~ lstat, data = Boston)
fit_full <- lm(medv ~ ., data = train)
summary(fit_simple)
summary(fit_full)

# Prediction
pred_test <- predict(fit_full, newdata = test)
```

The regression summary provides coefficient estimates, standard errors, t -tests, R^2 , adjusted R^2 and the overall F -test. Coefficients are interpreted while the other predictors are held constant.

6.1 Evaluate numerical predictions

$$\text{MSE} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} (y_i - \hat{y}_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}.$$

```
actual <- test$medv
mse <- mean((actual - pred_test)^2)
rmse <- sqrt(mse)
```

RMSE is expressed in the same unit as the response. A smaller test RMSE indicates better out-of-sample prediction for the same test set.

6.2 Recognise overfitting

```
pred_train <- predict(fit_full, newdata = train)
rmse_train <- sqrt(mean((train$medv - pred_train)^2))
rmse_test <- sqrt(mean((test$medv - pred_test)^2))
```

- Low training RMSE and similar test RMSE: the model generalises reasonably.
- Very low training RMSE but much larger test RMSE: possible overfitting.
- Both errors large: possible underfitting or weak predictors.

7 Stage 6: Diagnose the Linear Model

The assumptions describe the errors after the systematic predictor effects have been modelled.

Assumption	What to inspect	Desired result
Linearity	Scatterplot or residual pattern	No systematic curvature
Independence	Durbin–Watson test	Statistic near 2
Constant variance	Residuals versus fitted plot	Random band around zero
Residual normality	Q–Q plot and Shapiro test	Approximately straight Q–Q pattern
No serious multicollinearity	Variance inflation factors	No large VIF values

```
# Linearity
plot(Boston$lstat, Boston$medv)
abline(lm(medv ~ lstat, data = Boston))

# Residual diagnostics
plot(fitted(fit_full), residuals(fit_full))
abline(h = 0)
qqnorm(residuals(fit_full))
qqline(residuals(fit_full))
shapiro.test(residuals(fit_full))

# Independence and multicollinearity
lmtest::dwtest(fit_full)
car::vif(fit_full)
```

The lab uses $VIF > 5$ as a warning of substantial multicollinearity. High VIF values mean coefficient estimates may be unstable even when overall prediction remains acceptable.

Practical meaning. Assumption checking is not a ritual performed before fitting the model. Most regression assumptions concern the residuals, so they are examined after a model has been fitted.

8 Stage 7: Control Model Complexity

Adding predictors can improve training fit while making the model harder to interpret and more likely to overfit. Labs 03 and 04 introduce two solutions.

8.1 Subset and stepwise selection

The Hitters task predicts Salary after removing incomplete rows.

```
library(ISLR)
library(leaps)
data("Hitters")
Hitters <- na.omit(Hitters)

best <- regsubsets(Salary ~ ., data = Hitters, nvmax = 19)
best_sum <- summary(best)

cbind(RSS = best_sum$rss,
      R2 = best_sum$rsq,
      Cp = best_sum$cp,
      BIC = best_sum$bic,
      AdjR2 = best_sum$adjr2)

which.min(best_sum$cp)
which.min(best_sum$bic)
which.max(best_sum$adjr2)
```

The same framework supports:

```
forward <- regsubsets(Salary ~ ., data = Hitters,
                      nvmax = 19, method = "forward")
backward <- regsubsets(Salary ~ ., data = Hitters,
                       nvmax = 19, method = "backward")
```

- Best subset considers every model of each size.
- Forward selection starts small and adds predictors.
- Backward selection starts large and removes predictors.
- Minimum C_p or BIC and maximum adjusted R^2 indicate preferred sizes.

These criteria may select different models because they penalise complexity differently.

8.2 Ridge regression

Ridge estimates coefficients by minimising

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2.$$

It shrinks coefficients toward zero but usually does not set them exactly to zero.

```
x_train <- as.matrix(train[, setdiff(names(train), "mpg")])
y_train <- train$mpg
x_test  <- as.matrix(test[, setdiff(names(test), "mpg")])
y_test  <- test$mpg

cv_ridge <- cv.glmnet(x_train, y_train, alpha = 0)
lambda_ridge <- cv_ridge$lambda.min
ridge_fit <- glmnet(x_train, y_train, alpha = 0,
                    lambda = lambda_ridge)
pred_ridge <- predict(ridge_fit, newx = x_test)
rmse_ridge <- sqrt(mean((y_test - pred_ridge)^2))
```

8.3 LASSO

LASSO minimises

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j|.$$

Its absolute-value penalty can make some coefficients exactly zero, so LASSO performs estimation and feature selection together.

```
cv_lasso <- cv.glmnet(x_train, y_train, alpha = 1)
lambda_lasso <- cv_lasso$lambda.min
lasso_fit <- glmnet(x_train, y_train, alpha = 1,
                    lambda = lambda_lasso)

pred_lasso <- predict(lasso_fit, newx = x_test)
rmse_lasso <- sqrt(mean((y_test - pred_lasso)^2))

lasso_coef <- coef(cv_lasso, s = "lambda.min")
selected <- rownames(lasso_coef)[as.vector(lasso_coef) != 0]
```

8.4 Why cross-validation is central

The penalty λ controls model flexibility:

- small λ : weak shrinkage and a more flexible model;
- large λ : strong shrinkage and a simpler model.

`cv.glmnet()` estimates which value of λ predicts held-out folds best. The final choice between MLR, ridge and LASSO should compare their test RMSE values using the same split.

Key idea. Feature selection answers which predictors to retain. Regularisation answers how strongly their coefficients should be shrunk. Test performance determines whether the additional complexity is useful outside the training sample.

9 Stage 8: Change the Model When the Response Changes

9.1 Binary response: logistic regression

When $Y \in \{0, 1\}$, a linear model can predict values below 0 or above 1. Binary logistic regression models

$$\log\left(\frac{\pi}{1 - \pi}\right) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p,$$

where $\pi = P(Y = 1 \mid X)$. The predicted probability is

$$\pi = \frac{\exp(\eta)}{1 + \exp(\eta)}.$$

```
df <- read.table("leukemia_remission.txt", header = TRUE)

model_simple <- glm(REMISS ~ LI, data = df,
                    family = "binomial")
model_two <- glm(REMISS ~ LI + TEMP, data = df,
                 family = "binomial")
model_full <- glm(REMISS ~ ., data = df,
                  family = "binomial")
model_null <- glm(REMISS ~ 1, data = df,
                  family = "binomial")

summary(model_full)
confint(model_full)
exp(coef(model_full))
```

$\exp(\hat{\beta}_j)$ is an odds ratio:

- greater than 1: the odds of $Y = 1$ increase;
- less than 1: the odds decrease;
- equal to 1: no estimated change in odds.

Predicted probabilities can be converted into classes:

```
prob <- predict(model_full, type = "response")
pred <- ifelse(prob >= 0.50, 1, 0)
table(Actual = df$REMISS, Predicted = pred)
```

The Hosmer–Lemeshow test compares observed outcomes with fitted probabilities across risk groups:

```
ResourceSelection::hoslem.test(df$REMISS,
                               fitted(model_simple))
```

A p -value above 0.05 means that the test does not find strong evidence of poor fit. It does not prove that the model is perfect.

10 Stage 9: Compare Classification Models

Lab 06 uses the Iris species as an unordered three-class response.

10.1 LDA and QDA

LDA assumes the classes share a covariance matrix, producing linear decision boundaries. QDA allows a different covariance matrix for each class, producing more flexible quadratic boundaries.

```
set.seed(2222)
train_id <- sample(seq_len(nrow(iris)),
                  size = 0.8 * nrow(iris))
train <- iris[train_id, ]
test  <- iris[-train_id, ]

fit_lda <- MASS::lda(Species ~ ., data = train)
fit_qda <- MASS::qda(Species ~ ., data = train)

pred_lda <- predict(fit_lda, test)$class
pred_qda <- predict(fit_qda, test)$class
```

10.2 Naive Bayes

Naive Bayes combines class prior probabilities and predictor likelihoods:

$$P(C_k | \mathbf{x}) \propto P(C_k) \prod_{j=1}^p P(x_j | C_k).$$

Its simplifying assumption is conditional independence of predictors within each class.

```
fit_nb <- e1071::naiveBayes(Species ~ ., data = train)
pred_nb <- predict(fit_nb, newdata = test)
```

10.3 Confusion matrix and accuracy

```
conf <- table(Actual = test$Species, Predicted = pred_lda)
accuracy <- sum(diag(conf)) / sum(conf)
conf
accuracy
```

$$\text{Accuracy} = \frac{\text{number correctly classified}}{\text{total number classified}}.$$

Compare LDA, QDA and Naive Bayes on the same test observations. The confusion matrix shows which classes are confused, while accuracy gives one overall summary.

Practical meaning. The Naive Bayes code in Lab 06 predicts the same Iris observations used for fitting. That produces training accuracy and can be optimistic. For a fair model comparison, fit Naive Bayes on `train` and evaluate it on the same `test` set used for LDA and QDA.

11 Stage 10: Match the Model to Special Response Types

11.1 Multinomial logistic regression

Use multinomial logistic regression when the response has more than two categories with no natural order.

```
model_multi <- nnet::multinom(Species ~ ., data = iris)
summary(model_multi)
pred_multi <- predict(model_multi, newdata = iris)
```

Each set of coefficients describes a category's relative log-odds compared with the reference category. A positive coefficient increases those relative log-odds; a negative coefficient decreases them.

11.2 Ordinal logistic regression

Use ordinal logistic regression when categories have a meaningful order.

```
data("housing", package = "MASS")
model_ordinal <- MASS::polr(
  Sat ~ Infl + Type + Cont,
  weights = Freq,
  data = housing,
  Hess = TRUE
)
summary(model_ordinal)
```

`weights = Freq` represents the number of observations for each recorded combination. `Hess = TRUE` requests the Hessian matrix so that standard errors are available. The output contains predictor coefficients and thresholds separating adjacent ordered categories.

11.3 Poisson regression for counts

For a count response,

$$\log\{E(Y \mid X)\} = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p.$$

```
model_poisson <- glm(
  breaks ~ wool + tension,
  data = warpbreaks,
  family = poisson(link = "log")
)
summary(model_poisson)
exp(coef(model_poisson))
```

Exponentiated coefficients are incidence-rate ratios. A value of 1.20 means the expected count is multiplied by 1.20, holding other predictors constant.

11.4 Exponential regression through transformation

If

$$y = a \exp(bx),$$

then taking logarithms gives

$$\log(y) = \log(a) + bx.$$

This can be fitted with linear regression:

```
model_exp <- lm(log(rate) ~ conc, data = Puromycin)
a <- exp(coef(model_exp)[1])
b <- coef(model_exp)[2]
```

If $b > 0$, the fitted relationship increases exponentially; if $b < 0$, it decreases exponentially.

11.5 Gamma regression for positive continuous responses

For positive, right-skewed continuous responses, a Gamma model with a log link ensures a positive fitted mean:

```
model_gamma <- glm(
  Assault ~ Murder + UrbanPop + Rape,
  data = USArrests,
  family = Gamma(link = "log")
)
summary(model_gamma)
exp(coef(model_gamma))
```

Exponentiating a coefficient gives a multiplicative effect on the expected response. Predictors are interpreted while the others are held constant.

Model	Typical link	Meaning of exponentiated coefficient
Binary logistic	Logit	Odds ratio
Multinomial logistic	Generalised logit	Relative odds ratio
Ordinal logistic	Cumulative logit	Proportional odds effect
Poisson	Log	Incidence-rate ratio
Gamma	Log	Multiplicative mean effect

12 How the Early Skills Support Later Models

Early skill	What it teaches	Later use
Means and proportions	Translate a question into hypotheses	Interpret regression coefficient tests
Variance tests	Recognise unequal variability	Understand residual variance and class covariance
ANOVA	Compare several groups jointly	Understand categorical predictors and nested-model tests
Normality checks	Compare data with a reference distribution	Diagnose linear-model residuals
Correlation	Quantify a pairwise linear relationship	Anticipate association and multicollinearity
Confidence intervals	Express estimation uncertainty	Interpret coefficients and odds ratios

There are also important distinctions:

- Normality of a raw predictor is not a requirement of linear regression; normality is mainly considered for the residuals when conducting inference.
- A significant correlation does not mean a predictor remains significant after controlling for other predictors.
- A significant coefficient does not guarantee low prediction error.
- Good training performance does not guarantee good test performance.

13 A Practical End-to-End Example

Suppose the task is to predict vehicle fuel economy and identify a compact set of useful predictors.

Step 1: define the target. mpg is continuous, so linear regression is a suitable baseline.

Step 2: inspect the data.

```
cars <- read.csv("Cars Data.csv")
str(cars)
summary(cars)
colSums(is.na(cars))
```

Step 3: explore. Use histograms, scatterplots and correlations to understand distributions and pairwise relationships. Convert categorical variables to factors.

Step 4: split once.

```
set.seed(42)
id <- caret::createDataPartition(cars$mpg,
                                  p = 0.7,
                                  list = FALSE)
train <- cars[id, ]
test  <- cars[-id, ]
```

Step 5: fit a baseline and diagnose it.

```
mlr <- lm(mpg ~ ., data = train)
summary(mlr)
plot(fitted(mlr), residuals(mlr))
car::vif(mlr)
```

Step 6: regularise. Convert predictors to matrices, use cross-validation to choose λ , and fit ridge and LASSO.

Step 7: evaluate all models on the same test set.

```
rmse <- function(actual, predicted) {
  sqrt(mean((actual - predicted)^2))
}

rmse_mlr <- rmse(test$mpg, predict(mlr, test))
rmse_ridge <- rmse(y_test, pred_ridge)
rmse_lasso <- rmse(y_test, pred_lasso)

c(MLR = rmse_mlr,
  Ridge = rmse_ridge,
  LASSO = rmse_lasso)
```

Step 8: choose and explain. Prefer the model that balances test RMSE, stability and interpretability. If LASSO performs similarly while using fewer variables, its simpler model may be preferable. If ridge has the lowest test RMSE, it may be better for prediction even though it retains all predictors.

Key idea. The complete task is not finished when `summary(model)` is printed. A practical modelling answer must include data preparation, validation, diagnostics, performance and interpretation.

14 Master Model-Selection Guide

Situation	Method	Main output to report
One continuous response, one predictor	Simple linear regression	Slope, confidence evidence, R^2 , residual checks and prediction error
Continuous response, several predictors	Multiple regression	Adjusted effects, overall fit, assumptions and test RMSE
Many correlated predictors	Ridge	Cross-validated λ , coefficient shrinkage and test RMSE
Need automatic feature reduction	LASSO	Cross-validated λ , non-zero coefficients and test RMSE
Two-category response	Binary logistic	Odds ratios, fitted probabilities, fit test and classification results
Several unordered classes	LDA, QDA, Naive Bayes or multinomial logistic	Confusion matrix, accuracy and class-specific errors
Ordered response categories	Ordinal logistic	Predictor coefficients, threshold parameters and predicted classes
Count response	Poisson regression	Incidence-rate ratios and predicted counts
Positive right-skewed continuous response	Gamma regression	Multiplicative effects and fitted means

15 Common Mistakes to Avoid

1. **Using test data during training.** The test set should represent unseen data.
2. **Comparing models on different splits.** Use the same observations for a fair comparison.
3. **Evaluating on training data only.** Training performance is often optimistic.
4. **Treating correlation as causation.** Correlation only describes association.
5. **Interpreting logistic coefficients as probability changes.** Exponentiate them to obtain odds ratios.
6. **Ignoring the reference category.** Dummy-variable and logistic interpretations depend on it.
7. **Choosing a model only because its R^2 is larger.** Ordinary R^2 cannot decrease when predictors are added; consider adjusted R^2 , selection criteria and test error.
8. **Calling a non-significant Hosmer–Lemeshow result proof of fit.** It only means that the test did not detect clear lack of fit.
9. **Assuming 0.50 is always the best classification threshold.** The cost of false positives and false negatives should guide the threshold.
10. **Reporting an accuracy value without a confusion matrix.** The same accuracy can hide very different class-specific mistakes.

16 Reusable R Skeleton

```
head(dat)
str(dat)
summary(dat)
colSums(is.na(dat))
dat <- na.omit(dat)
dat$group <- factor(dat$group)
hist(dat$y)
plot(dat$x, dat$y)
cor.test(dat$x, dat$y)
set.seed(42)
id <- sample(seq_len(nrow(dat)), round(0.8 * nrow(dat)))
train <- dat[id, ]
test  <- dat[-id, ]
fit <- lm(y ~ ., data = train)
# or glm(y ~ ., data = train, family = "binomial")
summary(fit)
confint(fit)
pred <- predict(fit, newdata = test)
rmse <- sqrt(mean((test$y - pred)^2))
```

17 Final Practical Checklist

Before considering an analysis complete, verify that you can answer:

1. What is the response variable and what type of variable is it?
2. Which model matches that response type?
3. How were missing values and categorical variables handled?
4. Was the train-test split created before model comparison?
5. What is the simple baseline model?
6. Which assumptions or model conditions were checked?
7. Was model complexity controlled through selection or regularisation?
8. Which measure evaluates unseen-data performance?
9. What do the main coefficients mean in the original context?
10. What limitation should be stated with the final conclusion?

18 Core Formula Summary

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2},$$

$$\hat{Y}_{\text{linear}} = \hat{\beta}_0 + \sum_{j=1}^p \hat{\beta}_j X_j,$$

$$\text{logit}(\pi) = \log\left(\frac{\pi}{1-\pi}\right) = \beta_0 + \sum_{j=1}^p \beta_j X_j,$$

$$\pi = \frac{\exp(\eta)}{1 + \exp(\eta)},$$

$$OR_j = \exp(\beta_j),$$

$$\text{Accuracy} = \frac{\sum_k n_{kk}}{\sum_{j,k} n_{jk}},$$

$$\text{Ridge penalty} = \lambda \sum_{j=1}^p \beta_j^2,$$

$$\text{LASSO penalty} = \lambda \sum_{j=1}^p |\beta_j|,$$

$$\log\{E(Y \mid X)\} = \beta_0 + \sum_{j=1}^p \beta_j X_j \quad \text{for log-linked Poisson or Gamma models.}$$